



Sprawozdanie z projektu

Optymalizacja rozwiązywania Sudoku za pomocą algorytmu genetycznego

Skład zespołu:

Patryk Bogusławski Nr albumu 64906
Paweł Cabaj Nr albumu 65456

Warszawa 2025

Spis treści

1. Wstęp
2. Cel projektu
3. Opis wymagań systemu informacyjnego
 - 3.1. Zakres działania i funkcjonalności
 - 3.2. Ograniczenia systemu
4. Opis użytego algorytmu
 - 4.1. Charakterystyka algorytmu genetycznego
 - 4.2. Reprezentacja genotypu
 - 4.3. Inicjalizacja
 - 4.4. Funkcja oceny (fitness)
 - 4.5. Operatory genetyczne
 - 4.6. Strategia ewolucji
 - 4.7. Miary jakości optymalizacji
5. Szczegółowy opis implementacji
 - 5.1. Architektura systemu
 - 5.2. Moduły systemu
 - 5.3. Zasady projektowania
6. Interpretacja wyników
 - 6.1. Analiza wyników testów
 - 6.2. Wykresy wydajności
7. Przykład użycia
8. Wnioski
9. Źródła

1. Wstęp

Sudoku to klasyczna łamigłówka logiczna, której celem jest uzupełnienie planszy 9×9 cyframi od 1 do 9 w taki sposób, aby każda cyfra występowała tylko raz w każdym wierszu, kolumnie oraz w każdym z dziewięciu 3×3 podobszarów (bloków). Zagadka ta, pomimo pozornej prostoty, stanowi złożony problem kombinatoryczny, który może być efektywnie rozwiązywany przy użyciu algorytmów ewolucyjnych.

2. Cel projektu

Celem projektu jest opracowanie efektywnego algorytmu genetycznego do rozwiązywania łamigłówek Sudoku, z wizualizacją procesu i wyników. Projekt ma na celu zbadanie możliwości zastosowania metod genetycznych w rozwiązywaniu problemów kombinatorycznych oraz ocenę efektywności takiego podejścia w kontekście różnych poziomów trudności zagadek.

3. Opis wymagań systemu informacyjnego

3.1. Zakres działania i funkcjonalności

System realizuje następujące funkcjonalności:

1. **Generowanie losowych plansz Sudoku** o różnym poziomie trudności (kontrolowanym przez liczbę pustych pól)
2. **Rozwiązywanie plansz Sudoku** za pomocą algorytmu genetycznego z dostosowanymi operatorami
3. **Wizualizacja plansz** w formie konsolowej i graficznej z wykorzystaniem biblioteki matplotlib
4. **Pomiar wydajności** poprzez monitorowanie czasu wykonania i jakości rozwiązania
5. **Analiza konwergencji algorytmu** przez śledzenie postępu w kolejnych generacjach

3.2. Ograniczenia systemu

System posiada następujące ograniczenia:

1. **Rozmiar planszy:** Działa wyłącznie dla standardowego Sudoku o rozmiarze 9×9
2. **Środowisko wykonania:** Wymaga środowiska Google Colaboratory lub kompatybilnego
3. **Dostrajanie parametrów:** Może wymagać dostrojenia parametrów dla różnych poziomów trudności
4. **Czas wykonania:** Dla bardzo trudnych zagadek czas rozwiązania może być znaczący

4. Opis użytego algorytmu

4.1. Charakterystyka algorytmu genetycznego

Algorytm genetyczny zastosowany do rozwiązywania łamigłówki Sudoku stanowi adaptację klasycznych metod ewolucyjnych do specyficznych wymagań i ograniczeń tego problemu. Głównym celem jest znalezienie takiego układu liczb na planszy 9×9, który spełnia wszystkie zasady Sudoku:

1. Każdy wiersz musi zawierać cyfry od 1 do 9 bez powtórzeń
2. Każda kolumna musi zawierać cyfry od 1 do 9 bez powtórzeń
3. Każdy blok 3x3 musi zawierać cyfry od 1 do 9 bez powtórzeń
4. Liczby wpisane początkowo na planszy (tzw. "podpowiedzi") nie mogą być zmieniane

4.2. Reprezentacja genotypu

Struktura reprezentacji danych w algorytmie:

- **Gen:** Pojedynczy wiersz Sudoku (tablica 9 liczb)
- **Chromosom:** Kompletna plansza Sudoku (9 wierszy po 9 liczb)
- **Populacja:** Zbiór potencjalnych rozwiązań planszy Sudoku

4.3. Inicjalizacja

Algorytm rozpoczyna się od wygenerowania populacji początkowej, gdzie każdy chromosom (plansza) respektuje podpowiedzi z planszy początkowej i ma poprawnie zainicjalizowane wiersze. Każdy wiersz zawiera wszystkie cyfry od 1 do 9 bez powtórzeń, przy zachowaniu stałych pól z oryginalnej zagadki.

4.4. Funkcja oceny (fitness)

Funkcja oceny określa jakość danego rozwiązania przez zliczanie konfliktów w kolumnach i blokach 3x3:

- Dla każdej kolumny zliczane są duplikaty cyfr
- Dla każdego bloku 3x3 zliczane są duplikaty cyfr
- Wartość funkcji oceny to suma ujemnych wartości liczby duplikatów w kolumnach i blokach
- Optymalne rozwiązanie ma wartość fitness równą 0 (brak konfliktów)

4.5. Operatory genetyczne

Selekcja

Zastosowano selekcję turniejową:

- Losowo wybiera się grupę osobników (typowo 5)
- Z tej grupy wybierany jest osobnik z najwyższą wartością funkcji oceny
- Proces powtarza się do wybrania odpowiedniej liczby osobników

Krzyżowanie

Krzyżowanie uwzględnia specyfikę problemu Sudoku:

- Dla wierszy zawierających "podpowiedzi" stosuje się prosty wybór jednego z wierszy rodzicielskich
- Dla pozostałych wierszy stosuje się krzyżowanie jednopunktowe: część genów pobierana jest od pierwszego rodzica, a część od drugiego
- Prawdopodobieństwo krzyżowania wynosi standardowo 0.95

Mutacja

Zaimplementowano dwa typy mutacji:

- **Pełna mutacja:** Całkowite wygenerowanie nowego wiersza z zachowaniem podpowiedzi
- **Mutacja częściowa:** Zamiana miejscami dwóch losowych cyfr w wierszu, z pominięciem pozycji zawierających podpowiedzi

4.6. Strategia ewolucji

Algorytm implementuje strategię elitaryzmu, która gwarantuje, że najlepsze rozwiązanie z każdej generacji jest przenoszone do następnej generacji bez zmian. Zastosowano również wczesne zatrzymanie algorytmu:

- Jeśli przez 100 generacji nie ma poprawy najlepszego rozwiązania, algorytm kończy działanie
- Algorytm kończy działanie również, gdy znajdzie rozwiązanie idealne (fitness = 0)

4.7. Miary jakości optymalizacji

W algorytmie genetycznym do rozwiązywania Sudoku zastosowano następujące miary jakości optymalizacji:

- **Funkcja fitness:** Ocenia liczbę konfliktów w kolumnach i blokach 3x3. Wartość 0 oznacza rozwiązanie idealne (brak konfliktów). Im wartość bliższa 0, tym lepsze rozwiązanie.

- **Wskaźnik konwergencji:** Monitorowanie postępu przez śledzenie najlepszej wartości fitness w każdej generacji, historii najlepszych wartości fitness oraz licznika stagnacji.
- **Czas wykonania:** Mierzony od rozpoczęcia do zakończenia działania algorytmu, używany do porównania różnych konfiguracji parametrów.
- **Liczba generacji do znalezienia rozwiązania:** Miara szybkości konwergencji. Algorytm zapisuje, w której generacji znaleziono rozwiązanie. Mniejsza liczba generacji sugeruje bardziej efektywny algorytm.
- **Efektywność wczesnego zatrzymania:** Mechanizm zatrzymujący algorytm po 100 generacjach bez poprawy, co pozwala ocenić, jak szybko algorytm osiąga swoje plateau.

5. Szczegółowy opis implementacji

5.1 Architektura systemu

Kod jest zaprojektowany i jest oparty na obecnie powszechnym technikach, takich jak wielowątkowość i podejście modułarne. Wykorzystuje on powszechne operacje genetyczne, jednocześnie dbając o zachowanie warunków początkowych, co jest niezbędne dla uzyskania wyników poprawnych i za razem wiarygodnych. Jednocześnie ważne w algorytmie jest zapewnienie unikalności pól, które zostały już prawidłowo ustalone.

5.2 Moduły systemu

Program został podzielony na następujące moduły:

- **Wizualizacja:** Odpowiada za graficzne przedstawienie planszy Sudoku przed i po rozwiązaniu. Wykorzystuje bibliotekę matplotlib do tworzenia czytelnej wizualizacji siatki 9x9 z podziałem na mniejsze kwadraty 3x3.
- **Inicjacja rozwiązań:** Obejmuje metody tworzenia genów (pojedynczych wierszy), chromosomów (pełnych plansz) oraz populacji (zbioru wielu chromosomów) przy użyciu wielowątkowości dla wydajności.
- **Funkcja celu (dopasowanie):** Mierzy, jak dobrze dany chromosom spełnia warunki Sudoku pod kątem kolumn i bloków 3x3.
- **Operacje genetyczne:** Implementuje standardowe metody obliczeń genetycznych, takie jak krzyżowanie, mutacja i selekcja turniejowa.
- **Generowanie planszy Sudoku:** Używa rekurencyjnej funkcji do generowania pełnej, poprawnej planszy Sudoku, a następnie losowo wymazuje pola, tworząc zagadkę do rozwiązania.
- **Pętla główna algorytmu:** Implementuje klasyczny schemat algorytmu genetycznego: inicjalizacja, ocena, selekcja, krzyżowanie, mutacja, zamiana.

5.3 Zasady projektowania

Wyznacznikami przy przygotowaniu programu były następujące zasady:

- **Modularność:** Funkcje są wyraźnie oddzielone, co ułatwia modyfikację kodu w czasie pracy, debugowanie oraz prezentację kodu.
- **Wielowątkowość:** Przy generowaniu populacji, co poprawia wydajność.
- **Prawidłowe odwzorowanie zagadnienia:** Zachowanie pól statycznych podczas operacji genetycznych, co gwarantuje poprawność rozwiązania.
- **Dobra prezentacja wyników:** Czytelne przedstawienie planszy przed i po rozwiązaniu.
- **Dostęp do zmiany parametrów:** Można łatwo zmieniać rozmiar populacji, liczby pokoleń, prawdopodobieństwa mutacji i krzyżowania.
- **Prawidłowa generacja zagadki:** Zapewnia wygenerowanie poprawnej i unikalnej planszy Sudoku do rozwiązania.

6. Interpretacja wyników

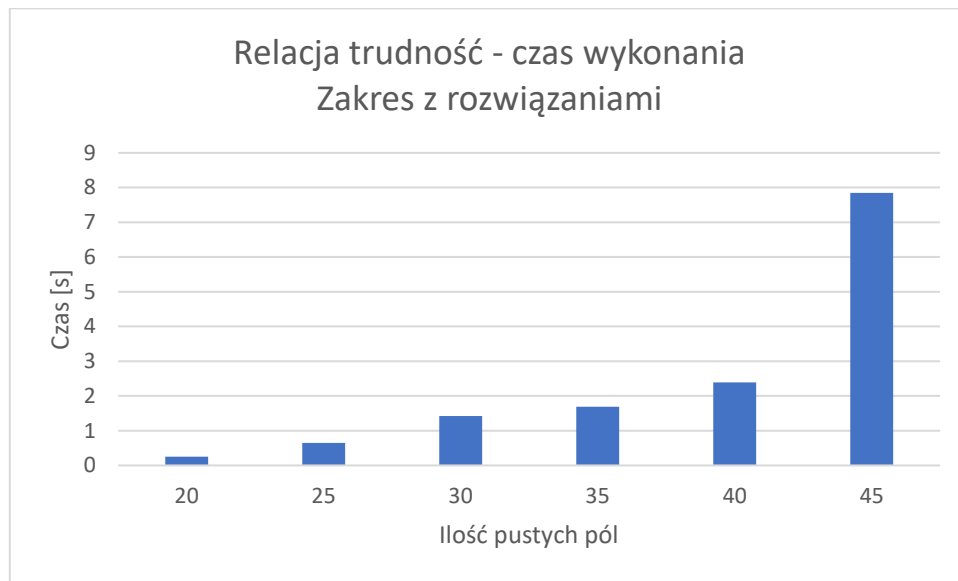
6.1. Analiza wyników testów

Na podstawie przeprowadzonych testów można stwierdzić, że wartość graniczna liczby pustych pól, determinująca trudność zagadki, mieści się w przedziale 45-50 pól do uzupełnienia. Poniżej przedstawiono wyniki pomiarów przeprowadzonych w środowisku Google Colaboratory:

Puste pola	Liczba pokoleń	Czas wykonania	Dopasowanie
20	2	0,25	0
25	6	0,65	0
30	12	1,42	0
35	15	1,69	0
40	33	2,39	0
45	115	7,85	0
50	181	13,60	-4
55	323	24,84	-4
60	352	27,08	-2
65	329	24,51	0
70	280	22,39	-7

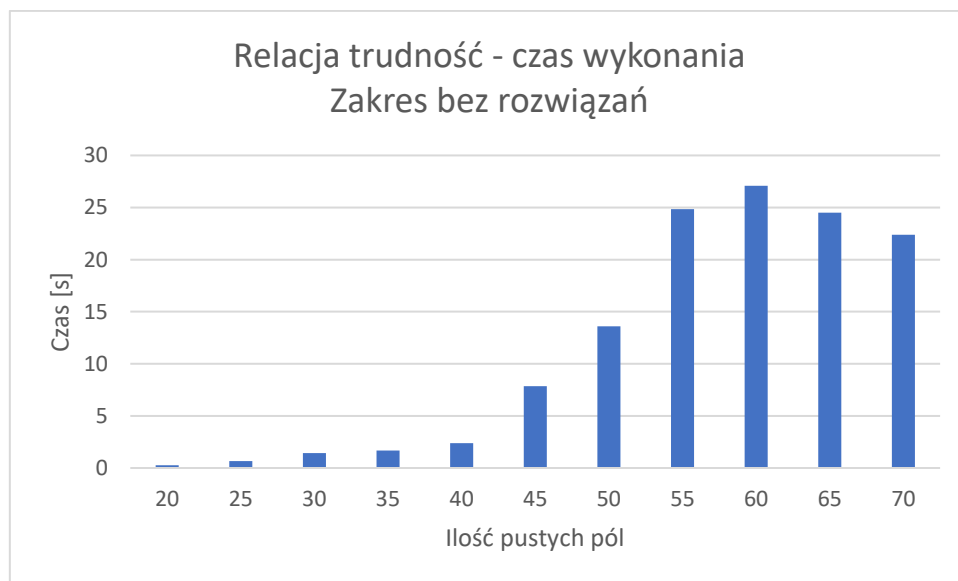
Tabela 1. Wyniki testów algorytmu genetycznego dla różnych poziomów trudności

6.2. Wykresy wydajności



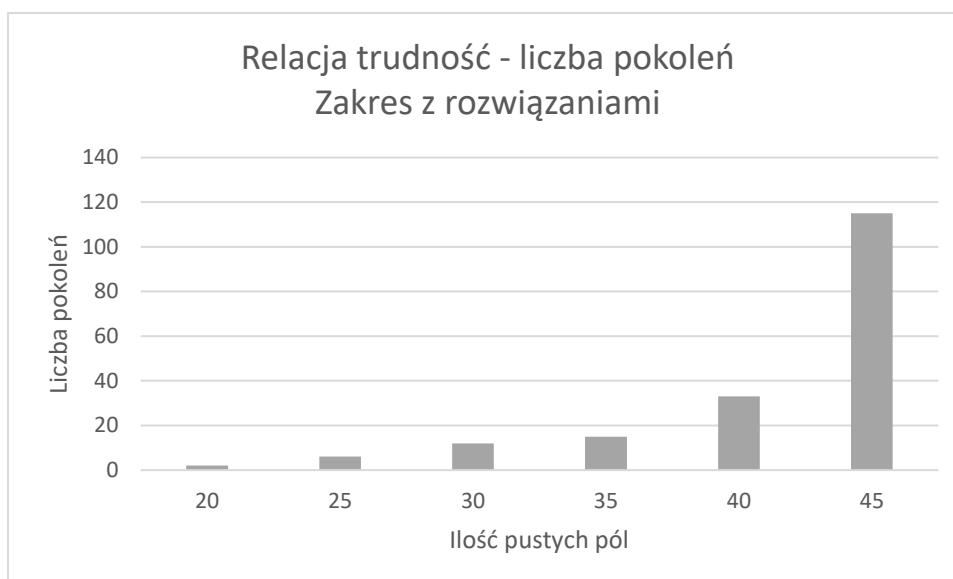
Rysunek 1. Relacja trudność - czas wykonania (zakres z rozwiązaniami)

Wykres przedstawia wykładniczy wzrost czasu wykonania algorytmu w zależności od liczby pustych pól. Dla zagadek o 20-45 pustych polach czas wykonania rośnie stopniowo od 0,25 do 7,85 sekundy.



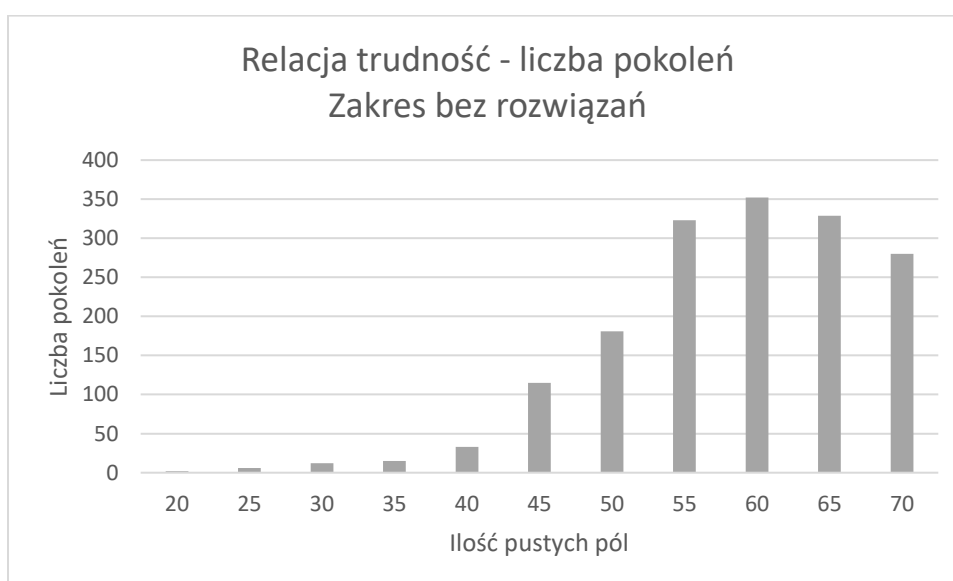
Rysunek 2. Relacja trudność - czas wykonania (zakres bez rozwiązań)

Dla zagadek przekraczających próg trudności (powyżej 50 pustych pól) czas wykonania stabilizuje się na poziomie około 22-27 sekund ze względu na mechanizm wczesnego zatrzymania.



Rysunek 3. Relacja trudność - liczba pokoleń (zakres z rozwiązaniami)

Wykres pokazuje wzrost liczby pokoleń potrzebnych do znalezienia rozwiązania. Dla łatwych zagadek (20 pustych pól) wystarczą 2 generacje, podczas gdy dla trudniejszych (45 pustych pól) potrzeba 115 pokoleń.



Rysunek 4. Relacja trudność - liczba pokoleń (zakres bez rozwiązań)

W przypadku zagadek, których algorytm nie może rozwiązać, liczba generacji osiąga maksymalną wartość około 300-350 ze względu na kryterium zatrzymania.

7. Przykład użycia

System został przetestowany na zagadce o 25 pustych polach. Informacje wyświetlane na konsoli obejmują:

- Parametry algorytmu (rozmiar populacji, prawdopodobieństwo mutacji, krzyżowania)
- Postęp obliczeń co 100 generacji
- Końcowe wyniki: liczbę generacji, czas wykonania, wartość fitness
- Wizualizację planszy przed i po rozwiązaniu

Przykład komunikatu na konsoli:

Uruchomienie algorytmu...

Numer pokolenia 0: dopasowanie = -10

Numer pokolenia 6: dopasowanie = 0

Znaleziono rozwiązanie!

Czas wykonywania: 0.86 seconds

Wartość najlepszego dopasowania: 0

Plansza – zagadka

Plansza - zagadka								
5	7	4	3		9		2	
1	6	2	8		5		7	
3		8		7				4
8	2	3	9	6	7		4	
7	4		1	5	8	9	3	2
	1	5	4	3			6	8
4		9	7			6	5	3
6	3	7	5	8	4	2		1
		1		9	3	4		7

Rozwiązanie

Rozwiązanie								
5	7	4	3	1	9	8	2	6
1	6	2	8	4	5	3	7	9
3	9	8	2	7	6	5	1	4
8	2	3	9	6	7	1	4	5
7	4	6	1	5	8	9	3	2
9	1	5	4	3	2	7	6	8
4	8	9	7	2	1	6	5	3
6	3	7	5	8	4	2	9	1
2	5	1	6	9	3	4	8	7

8. Wnioski

Projekt potwierdził, że algorytmy genetyczne mogą być skutecznym narzędziem do rozwiązywania problemów kombinatorycznych, takich jak Sudoku. Implementacja wykazała następujące wnioski:

- Algorytm genetyczny jest szczególnie efektywny dla zagadek o średnim poziomie trudności.
- Dla bardziej złożonych zagadek czas wykonania znacząco rośnie, co sugeruje potrzebę optymalizacji lub zastosowania hybrydowych podejść.
- Strategia elitaryzmu i wczesnego zatrzymania okazała się skuteczna w zapobieganiu niepotrzebnym obliczeniom.
- Specyficzne operatory genetyczne dostosowane do problemu Sudoku (zachowanie stałych pól) są kluczowe dla osiągnięcia poprawnych wyników.
- Wizualizacja procesu i wyników znacznie ułatwia analizę i debugowanie algorytmu.

9. Źródła

<https://github.com/MojTabaa4/genetic-algorithm>

<https://nidragedd.github.io/sudoku-genetics/>